

1. Idea Principal del Agente

Minimaxer implementa el algoritmo Minimax con poda Alpha-Beta, un algoritmo clásico en juegos por turnos. En cada turno, el agente construye un árbol de juego simulando todas las secuencias posibles de movimientos hasta una profundidad $depth$, asumiendo que el oponente también jugará óptimamente. La poda Alpha-Beta elimina ramas que nunca pueden influir en la decisión final, reduciendo drásticamente el costo computacional sin sacrificar calidad.

Componentes clave:

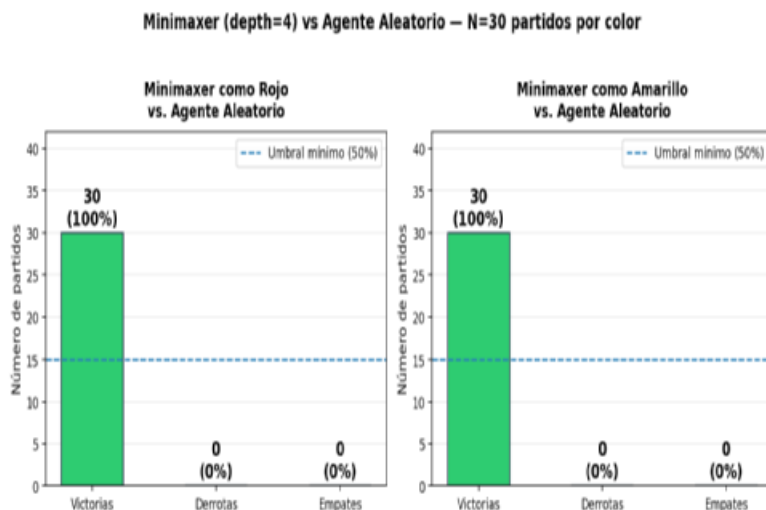
Componente	Descripción
<code>score_window()</code>	Puntúa una ventana de 4 celdas: +5 (3 propias+vacía), -4 (3 rivales+vacía), +2 (2 propias+2 vacías)
<code>heuristic()</code>	Suma puntajes de todas las ventanas horizontales, verticales y diagonales + bonus por columna central
<code>minimax()</code>	Búsqueda recursiva con poda α - β . Alterna nodo maximizador (propio) y minimizador (oponente)
<code>act()</code>	Detecta color, aplica corto-circuito victoria/bloqueo inmediato, luego invoca minimax
<code>depth</code>	Parámetro numérico configurable: profundidad de búsqueda del árbol (default $t=5$, torneo= 4)

2. Análisis del Agente

Se evaluaron dos configuraciones del agente ($depth=4$ como versión final, $depth=2$ como versión reducida) contra el agente aleatorio y en auto-juego. Todos los experimentos se repitieron para ambos colores (Rojo y Amarillo).

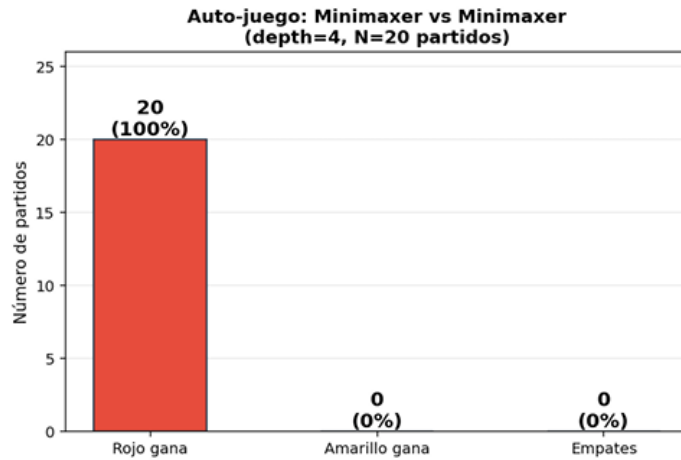
Datos de los Gráficos y Experimentos:

- Minimaxer como Rojo vs. Agente Aleatorio: 30 Victorias (100%), 0 Derrotas (0%), 0 Empates (0%). (*Umbral mínimo 50%*).
- Minimaxer como Amarillo vs. Agente Aleatorio: 30 Victorias (100%), 0 Derrotas (0%), 0 Empates (0%). (*Umbral mínimo 50%*).



- Mirrormatch (autojuego): Minimaxer vs Minimaxer ($depth=4$, $N=20$ partidos)

- Rojo gana: 20 (100%).
- Amarillo gana: 0 (0%).
- Empates: 0 (0%).
- Fig. 2 - Auto-juego: Minimaxer vs Minimaxer (\$depth=4, N=20\$).



Conclusiones del análisis:

- Minimaxer gana el 100% de los partidos contra el aleatorio con ambos colores, superando el umbral mínimo del 50%.
- En mirrormatch, Rojo gana sistemáticamente: la ventaja del primer movimiento no está compensada en la heurística actual.

3. Propuestas de Mejora

- Heurística asimétrica por color (cuello de botella: Fig. 2): La función heurística usa los mismos pesos para ambos colores, sin compensar la ventaja posicional del primer jugador. Añadir pesos diferenciados por color equilibraría el mirrormatch. Impacto esperado: reducir la ventaja de Rojo de 100% a ~50-60%.